LLM Application & Model Lifecycle Management

Industry Standards & Best Practices for LLMOps

PHASE 1: Planning & Requirements

Key Activities:
- Define use case & business objectives
- Identify stakeholders & requirements
- Select evaluation metrics (BLEU, ROUGE, perplexity)
- Determine model size & architecture
- Establish budget & resource constraints

Why: Clear requirements prevent scope creep and align technical choices with business value

Solution: Use CRISP-DM framework adapted for LLMs; create project charter with clear success criteria

PHASE 2: Data Collection & Preparation

Key Activities:
- Curate training datasets (quality > quantity)
- Implement data versioning (DVC, LakeFS)
- Clean & preprocess data
- Create prompt templates & few-shot examples
- Build evaluation datasets
- Address bias & toxicity in data
- Establish data governance policies

Why: Data quality directly impacts model performance; versioning enables reproducibility and debugging

Solution: Implement DVC for data versioning; use tools like Cleanlab for data quality; create diverse, representative datasets following principles from "Data-Centric AI"

PHASE 3: Model Selection & Development

Key Activities:
- Select base model (GPT-4, Claude, Llama, etc.)
- Choose development approach:
  - Zero-shot / Few-shot prompting
  - RAG (Retrieval Augmented Generation)
  - Fine-tuning (LoRA, QLoRA, full fine-tuning)
- Design prompt engineering strategy
- Implement guardrails & safety measures
- Create model registry (MLflow, Weights & Biases)
- Version control prompts & configurations

Why: Right approach reduces costs & time-to-market; model registry ensures reproducibility & compliance

Solution: Start with prompting, move to RAG if knowledge needed, fine-tune only if necessary; use MLflow for tracking; implement A/B testing framework early

PHASE 4: Experimentation & Training

Key Activities:
- Run controlled experiments (track all hyperparameters)
- Monitor training metrics (loss, perplexity)
- Implement early stopping & checkpointing
- Optimize for efficiency (quantization, pruning)
- Validate on held-out test sets
- Conduct human evaluation & RLHF
- Document experiment results

Why: Systematic experimentation prevents overfitting; tracking enables learning from failures and reproducing successes

Solution: Use Weights & Biases or MLflow for experiment tracking; implement automated evaluation pipelines; use stratified sampling for test sets

PHASE 5: Evaluation & Validation

Key Activities:
- Automated metrics (BLEU, ROUGE, BERTScore)
- Task-specific evaluation (accuracy, F1, etc.)
- Human evaluation (quality, relevance, safety)
- Bias & fairness testing
- Adversarial testing & red teaming
- Safety benchmarks (TruthfulQA, ToxiGen)
- Performance benchmarking (latency, throughput)
- Cost analysis
- Model card creation

Why: Multi-faceted evaluation catches issues metrics miss; model cards ensure transparency & responsible AI practices

Solution: Combine automated metrics with human evaluation; use frameworks like HELM or lm-evaluation-harness; establish evaluation cadence aligned with deployment frequency

Feedback Loop

MODEL LIFECYCLE MANAGEMENT

Model Versioning

Components:
- Semantic versioning (Major.Minor.Patch)
- Git-based model versioning
- Model registry (MLflow, W&B)
- Metadata tracking
- Lineage tracking

Industry Standard:
MLflow Model Registry, DVC

Why: Enables rollback, A/B testing, audit trails

Solution: Tag models with Git SHA + training date; store in centralized registry

Model Registry & Governance

Components:
- Centralized model repository
- Approval workflows
- Access controls (RBAC)
- Compliance documentation
- Model cards & datasheets

Industry Standard:
Model cards, SOC 2 compliance

Why: Ensures regulatory compliance, prevents unauthorized deployments

Solution: Implement stage gates (dev→staging→prod); require sign-offs for production

Model Monitoring

Metrics to Track:
- Performance metrics (accuracy, latency)
- Input/output distributions
- Prediction drift
- Data quality metrics
- Cost per inference

Industry Standard:
Prometheus, Grafana, Arize AI

Why: Early detection of degradation prevents user impact

Solution: Set up dashboards with alerts; implement automated retraining triggers

Drift Detection

Types of Drift:
- Concept drift (P(Y|X) changes)
- Data drift (P(X) changes)
- Label drift (P(Y) changes)

Detection Methods:
- Statistical tests (KS test, PSI)
- Embedding distance metrics

Industry Standard:

Why: Models degrade as world changes; proactive detection saves reputation

Solution: Monitor weekly; retrain when drift threshold exceeded; maintain champion/challenger setup

Continuous Learning

Strategies:
- Automated retraining pipelines
- Active learning (query hard examples)
- Human-in-the-loop feedback
- A/B testing new versions
- Incremental learning

Industry Standard:
Kubeflow Pipelines, Airflow

Why: Keeps models fresh; reduces manual intervention; improves ROI

Solution: Schedule retraining (weekly/monthly); use shadow mode for validation; gradual rollout

Model Retirement

Process:
- Deprecation notices (to users/teams)
- Grace period for migration
- Archive model artifacts
- Document reasons for retirement
- Clean up infrastructure

Industry Standard:
12-week deprecation timeline

Why: Reduces maintenance burden; prevents zombie models; controls costs

Solution: Define retirement criteria (performance, usage, age); automate cleanup scripts

CROSS-CUTTING CONCERNS

🔒 Security & Privacy

- Input sanitization & prompt injection defense
- API authentication (OAuth, API keys)
- Data encryption (at rest & in transit)
- PII detection & redaction
- Model access controls

Standards: OWASP Top 10 for LLMs, NIST AI Risk Framework

Solution: Use LLM firewalls (NeMo Guardrails, Guardrails AI); implement content filtering

📊 Observability & Logging

- Distributed tracing (OpenTelemetry)
- Prompt & completion logging
- Performance metrics (p50, p95, p99 latency)
- Error tracking & alerting
- Cost tracking per request

Standards: OpenTelemetry, Datadog, Langfuse

Solution: Integrate OpenTelemetry early; use structured logging; set up SLO dashboards

💰 Cost Management

- Token usage tracking & budgeting
- Caching strategies (semantic cache, KV cache)
- Request batching & rate limiting
- Model size optimization (quantization)
- Smart routing (GPT-4 vs GPT-3.5)

Standards: FinOps for ML

Solution: Implement prompt compression; use smaller models for simple tasks; set per-user quotas

⚖️ Responsible AI

- Bias testing & mitigation
- Explainability (chain-of-thought, citations)
- Content moderation
- Hallucination detection
- Ethical guidelines & use policies

Standards: EU AI Act, Model Cards, Datasheets

Solution: Regular bias audits; use fact-checking APIs; maintain prohibited use cases list

📖 Legend

- Why: Rationale & Benefits
- Solution: Implementation
- Sequential Flow
- Feedback Loop

Industry standards referenced throughout follow MLOps best practices from Google, Microsoft, AWS, and emerging LLMOps frameworks.